Informatics Institute of Technology

In Collaboration with

Robert Gordon University, Aberdeen, Scotland

AI-Driven Dynamic Narrative Personalization in Games:

A Closed-Loop Plugin Using Hexad Player Modeling, RAG-Grounded LLM Generation, and PPO Reinforcement Learning

Final Year Thesis by

Rivindu Bopage

RGU ID – 2312553

IIT ID – 20220580

Supervised by

[Supervisor Name]

April 2026

Submitted in partial fulfilment of the requirements for the
BSc. (Hons) in Artificial Intelligence and Data Science at the Robert Gordon University, Aberdeen, UK.

DECLARATION

I confirm that the work contained in this BSc project report has been composed solely by myself and has not been accepted in any previous application for a degree. All sources of information have been specifically acknowledged and all verbatim extracts are distinguished by quotation marks.

Date: ______________________________

Student's Name: ______________________________

The above student carried out his research project under my supervision.

Date: ______________________________

Supervisor's Name: ______________________________

SPER FORM

[Insert signed SPER Ethics Form here. Use `\includepdf` to embed the signed PDF.]

ABSTRACT

Narrative personalization in games — the capacity of a game's story to adapt meaningfully to individual player behaviour — remains an under-explored intersection of artificial intelligence, human–computer interaction, and game design. Most commercial games deliver identical narrative content regardless of how a player engages with the world, foregoing the well-established link between personalized experience and intrinsic motivation.

This dissertation presents the design, implementation, and evaluation of a closed-loop Unity plugin for real-time dynamic narrative personalization. The system integrates five tightly coupled modules: (1) a telemetry collector that streams 23 behavioural signals from the game client every five seconds; (2) a player modelling service that derives an 11-feature representation, a six-dimensional continuous Hexad player-type profile, and a Flow state classification (FLOW, BOREDOM, ANXIETY, APATHY) from those signals; (3) a narrative generation service that employs Retrieval-Augmented Generation (RAG) over a structured lore corpus using the sentence-transformers all-MiniLM-L6-v2 model together with an episodic session memory store; (4) an adaptation engine based on Proximal Policy Optimization (PPO), trained via Stable-Baselines3 over a custom Gymnasium Markov Decision Process environment; and (5) a Unity Content Injection Module that surfaces the generated narrative inside a 2D top-down RPG testbed.

A key novelty is the derivation of Hexad player-type scores entirely from gameplay telemetry — without requiring the player to complete a questionnaire — enabling continuous, within-session profiling. The PPO agent selects among eight narrative actions (e.g., PROVIDE_GUIDANCE, ADD_MYSTERY, INCREASE_URGENCY) based on the current Flow state and Hexad profile, with a reward function shaped around challenge–skill balance. A cold-start heuristic governs the first 120 seconds before the learned policy activates.

The system is evaluated through a between-subjects user study ($N = 50$; 25 Experimental, 25 Control). The primary dependent variable is the Flow subscale of the Game Experience Questionnaire (GEQ); secondary measures include the GEQ Immersion subscale, the custom three-item Narrative Personalization Scale (NPS-3), and the miniPXI player experience inventory. Qualitative data are gathered through semi-structured interviews analysed using Reflexive Thematic Analysis (Braun and Clarke, 2022). Results are pending collection and are presented as structured placeholders.

The work makes six original contributions: telemetry-derived Hexad profiling, RAG-grounded narrative generation, episodic session memory in the LLM prompt, PPO for narrative action selection, a Flow-based MDP reward, and a complete closed-loop integration of all five modules within a single game plugin.

Keywords: narrative personalization, player modelling, Hexad typology, Flow theory, reinforcement learning, large language models, retrieval-augmented generation, game experience, Unity plugin.

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, [Supervisor Name], for their invaluable guidance, constructive feedback, and consistent support throughout the duration of this research project. Their expertise and encouragement were instrumental in shaping the direction of this work.

I extend my thanks to the academic staff at Informatics Institute of Technology and Robert Gordon University for providing the academic foundation and resources that made this project possible. I am also grateful to the participants who volunteered their time for the user study; their contributions are central to the evaluation of this work.

I would like to acknowledge the open-source communities behind the tools that underpin this system — FastAPI, Stable-Baselines3, Gymnasium, Ollama, and sentence-transformers — whose freely available, well-documented software made this research feasible within the scope of an undergraduate project.

Finally, I thank my family and friends for their patience, understanding, and encouragement throughout this academic year.

Contents

List of Figures

List of Tables

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
DDA	Dynamic Difficulty Adjustment
DPM	Damage Per Minute
DV	Dependent Variable
GEQ	Game Experience Questionnaire
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
MDP	Markov Decision Process
miniPXI	Mini Player Experience Inventory
NPC	Non-Player Character
NPS-3	Narrative Personalization Scale (3-item)
PPO	Proximal Policy Optimization
RAG	Retrieval-Augmented Generation
RL	Reinforcement Learning
RPG	Role-Playing Game
SDT	Self-Determination Theory
SQL	Structured Query Language
TMPro	TextMeshPro
UI	User Interface
UPM	Unity Package Manager
WS	WebSocket

Chapter 1

INTRODUCTION

1.1 Chapter Overview

This chapter introduces the research by outlining its central focus, motivation, and purpose. It begins with the background and context for narrative personalization in games, identifies the specific problem addressed, presents the research questions and objectives, discusses the significance and scope of the work, and concludes with an outline of the thesis structure. The chapter establishes the foundation upon which the subsequent literature review, methodology, implementation, and evaluation chapters build.

1.2 Background and Motivation

Games occupy a unique position in interactive media: they are the only narrative form in which the audience simultaneously controls the protagonist and shapes the story's unfolding. Yet despite the enormous expressive potential this interactivity affords, the vast majority of commercially released titles deliver largely identical narrative sequences to every player. The village elder delivers the same warning, the antagonist delivers the same monologue, and the quest log records the same objectives regardless of whether the player is a cautious explorer who reads every piece of environmental lore or an aggressive disruptor who skips dialogue and charges headlong into combat.

This homogeneity is not a result of disinterest from game designers — narrative branching is a well-recognized craft challenge — but rather a structural consequence of the cost of authoring alternatives. Every branch, every personalized response, requires writing, voice-acting, and integration effort that scales combinatorially. The practical ceiling for authored branching in even the most narrative-ambitious titles (e.g., CD Projekt Red's *Cyberpunk 2077* or Larian Studios' *Baldur's Gate 3*) is reached long before true player-adaptive personalization becomes possible.

The emergence of large language models (LLMs) capable of coherent long-form natural language generation, combined with advances in real-time player behaviour modelling and reinforcement learning-based policy optimization, suggests a new possibility: narrative that is *generated*, rather than authored, in response to the individual player's demonstrated preferences, cognitive state, and engagement patterns (Brown et al., 2020). If such a system can be made reliable, grounded in lore-consistent content, and responsive to psychological signals of engagement, it could represent a qualitative shift in the nature of player experience.

The motivation for this research is threefold. First, motivation research in psychology —

particularly Self-Determination Theory (Ryan and Deci, 2000) — demonstrates that perceived competence, autonomy, and relatedness are foundational to sustained intrinsic engagement, and narrative personalization is a direct mechanism for signalling each of these. Second, the commercial games industry has invested heavily in player analytics and telemetry infrastructure, yet this data is used almost exclusively for monetization and retention metrics rather than for real-time content adaptation. Third, the convergence of accessible RL frameworks (Stable-Baselines3; Raffin et al., 2021), open-weight LLMs (Phi-3.5 Mini; Microsoft, 2024), and semantic search tools (sentence-transformers; Wang et al., 2020) creates a technical environment in which building such a system is feasible within the scope of an undergraduate research project.

This dissertation takes up that challenge. It presents a closed-loop plugin — implementable within Unity — that continuously profiles the player using a telemetry-derived Hexad player-type model (Tondello et al., 2016) and a Flow state classifier grounded in Csikszentmihalyi's (1990) challenge–skill balance theory. A Proximal Policy Optimization (PPO; Schulman et al., 2017) agent selects among eight narrative modulation actions based on these real-time signals. A Retrieval-Augmented Generation (RAG) pipeline grounds the LLM's output in a structured lore corpus, and an episodic session memory component inspired by EM-LLM (Fountas et al., 2024) ensures narrative continuity across a session. The resulting system is evaluated against a static narrative control in a between-subjects user study, measuring Flow experience, immersion, and perception of personalization using established psychometric instruments.

1.3 Problem Statement

The central problem this research addresses is the following: **current game narrative systems are static in the sense that they do not adapt their tone, urgency, or content to the psychological state and demonstrated preferences of the individual player in real time.** This creates a mismatch between player engagement state and narrative offering. A player experiencing BOREDOM — whose challenge–skill ratio has fallen below a meaningful threshold — continues to receive the same measured, low-stakes descriptions that a Flow-state player receives. A player in ANXIETY — overwhelmed by challenge — receives no additional guidance from the story world. A player whose behaviour reveals deep curiosity about lore receives the same minimal world description as one who ignores it entirely.

The consequences of this mismatch are reduced engagement, diminished immersion, and missed opportunities to use narrative as a lever for returning players to a state of optimal experience. The problem is not merely aesthetic: motivation research (Ryan and Deci, 2000) demonstrates that perceived competence, autonomy, and relatedness are foundational to sustained intrinsic engagement, and narrative personalization is a direct mechanism for signalling each of these.

The challenge of solving this problem is threefold. First, reliable real-time player state inference requires robust signal processing from noisy behavioural telemetry. Second, LLM-generated narrative in games must be tightly constrained to avoid breaking fictional immersion through anachronism, fourth-wall violations, or lore inconsistency. Third, the adaptation policy must be learned rather than fully hand-crafted, because the mapping from player state to optimal narrative action is complex and context-dependent.

1.4 Research Gap

Existing approaches to narrative personalization in games fall into three broad categories, each of which leaves significant gaps that the present work addresses.

First, *authored branching systems* (e.g., Mass Effect, The Witcher series, Disco Elysium) provide the illusion of personalization through meaningful choices but are ultimately deterministic in their narrative delivery relative to choice history. These systems do not adapt to behavioural signals such as play pace, combat avoidance, or exploration rate — they adapt only to explicit player-authored choices within a pre-specified decision space.

Second, *procedural narrative generation systems*, such as Tracery (Compton et al., 2015) and the Expressionist framework (Kreminski et al., 2019), demonstrated that grammatical expansion could produce contextually varied textual content. However, such systems still required extensive authoring of production rules and lacked the capacity to respond to the player's psychological state.

Third, *LLM-based narrative generation systems* such as PANGeA (Todd et al., 2024) have demonstrated that LLMs can produce coherent, in-world narrative content when given sufficient contextual grounding. However, PANGeA does not integrate a real-time player model or an adaptive policy; its generation is procedural (driven by game state) rather than personalized (driven by player psychological state). Similarly, LIGS (CHI 2025) investigated LLM integration into game narrative pipelines with a focus on coherence, but without player modelling or RL-based adaptation.

No prior work has integrated continuous Hexad profiling derived from behavioural telemetry, Flow-based RL reward shaping, RAG-grounded LLM generation, and episodic session memory into a single closed-loop system. The present work addresses this gap directly.

1.5 Research Questions

This dissertation is organized around three research questions:

RQ1: Can a closed-loop AI system driven by continuous Hexad profiling and Flow-state detection produce narrative that is perceived as more personalized than static pre-authored narrative?

RQ2: Does PPO-driven narrative adaptation result in measurably higher Flow experience (as measured by the GEQ Flow subscale) compared to a control condition receiving static narrative?

RQ3: How do players perceive and experience dynamically personalized narrative in terms of engagement and immersion?

RQ1 addresses the subjective perception of personalization. RQ2 targets the objective psychological measure of Flow, operationalized through a validated instrument. RQ3 is exploratory and captures nuances that quantitative measures may not surface.

1.6 Research Aim and Objectives

The overarching aim of this research is to demonstrate that a closed-loop, AI-driven narrative personalization system embedded within a game engine plugin can measurably improve player experience relative to static narrative delivery.

The specific objectives are:

1. To design and implement a five-module pipeline that integrates telemetry collection, player modelling, narrative generation, RL-based adaptation, and in-game content injection within a single coherent system.
2. To develop a method for deriving a six-dimensional Hexad player-type profile from behavioural telemetry without requiring a player questionnaire.
3. To construct a Gymnasium-compatible MDP environment that models narrative adaptation as a sequential decision problem with a Flow-based reward signal.
4. To build a RAG pipeline grounded in a structured game lore corpus that constrains LLM output to in-world content.
5. To design and conduct a between-subjects user study ($N = 50$) using validated psychometric instruments to evaluate the system's impact on Flow, immersion, and perceived personalization.
6. To contribute qualitative insight through reflexive thematic analysis of post-play interviews.

1.7 Significance of the Research

This research is significant for several reasons. First, it demonstrates the feasibility of integrating multiple AI techniques — player modelling, reinforcement learning, retrieval-augmented generation, and episodic memory — into a single, deployable game plugin. This integration has not been achieved in prior work and represents a practical contribution to the game development community.

Second, the telemetry-derived Hexad profiling method addresses a practical barrier to player-type-adaptive game design: the requirement for pre-play questionnaires that disrupt the player experience and provide only static, one-time snapshots. By inferring player types from behavioural data collected during gameplay, the system enables continuous, unobtrusive profiling that updates as the player's engagement patterns evolve.

Third, the use of PPO reinforcement learning for narrative action selection — rather than mechanical difficulty adjustment — extends the RL-for-games paradigm into the narrative domain. This reframing demonstrates that narrative tone and content selection can be formalized as a sequential decision problem amenable to policy optimization, opening a new design space for adaptive game narratives.

Fourth, the evaluation design provides a rigorous mixed-methods framework — combining validated psychometric instruments (GEQ, miniPXI) with qualitative thematic analysis — that can serve as a template for future studies evaluating AI-generated game content.

Finally, the work contributes to the broader discourse on AI and player experience by demonstrating that AI-driven personalization can operate within the latency and quality constraints of real-time gameplay, using only locally hosted, open-weight models without dependence on cloud services.

1.8 Scope of the Research

The scope of this work is bounded in several important respects. The testbed is a purpose-built 2D top-down RPG rather than a commercially released game; this ensures experimental control but limits ecological validity. The Hexad profiling method is rule-based and heuristic rather than a trained classifier; this is a deliberate design choice motivated by the cold-start constraint (no pre-existing player data) but introduces approximation error relative to ground-truth questionnaire-derived types.

The LLM employed is Phi-3.5 Mini (3.8B parameters), a locally hosted model via Ollama. This choice prioritizes privacy and latency predictability over raw generation quality; larger cloud-hosted models would likely yield more fluent output but introduce dependency, cost, and data governance concerns. The PPO agent is trained on a simulated environment rather than directly on human play data, which means the learned policy reflects the simulator's transition dynamics rather than empirical player behaviour.

The user study is bounded to $N = 50$, which is adequate for detecting medium-to-large effect sizes ($d \geq 0.5$) but would be insufficient to detect subtle effects. The study uses a between-subjects design, which eliminates carry-over effects but requires careful matching of conditions for comparable baseline experience. Results from Chapter 5 onward are presented as structured templates pending data collection.

1.9 Thesis Structure

The remainder of this thesis is structured as follows. Chapter 2 reviews the relevant literature across narrative personalization, player modelling, Flow theory, LLMs for games, reinforcement learning for adaptation, and episodic memory. Chapter 3 presents the research methodology in detail, covering the system architecture, each module's design, the testbed game, and the evaluation design including hypotheses, instruments, procedure, and analysis plan. Chapter 4 documents the technical implementation, including the technology stack, backend service, database layer, and Unity plugin. Chapter 5 presents the results of the user study (as structured placeholders pending data collection). Chapter 6 discusses the findings, contributions, limitations, and threats to validity. Chapter 7 concludes with a summary of contributions, future work directions, a reflective account of the research process, and closing remarks. Full questionnaire instruments, the API specification, and the MDP formal specification are provided in the appendices.

Chapter 2

LITERATURE REVIEW

2.1 Chapter Overview

This chapter critically reviews the existing literature across six domains that inform the design of the proposed system: narrative personalization in games, player modelling approaches, Flow theory in game design, large language models for game narrative, reinforcement learning for game adaptation, and episodic memory for LLMs. For each domain, the chapter examines key contributions, identifies methodological strengths and limitations, and positions the current work relative to the state of the art. The chapter concludes with a synthesis of the research gap that this dissertation addresses, supported by a comparative table of related work.

2.2 Narrative Personalization in Games

The question of how games might deliver individualized narrative experiences has been explored for several decades. Early work on interactive narrative focused on structural approaches: drama management systems such as the Façade project (Mateas and Stern, 2003) attempted to guide story structure toward dramatically satisfying outcomes by monitoring player actions and intervening in the plot. These systems were computationally tractable but brittle — they required exhaustive hand-authoring of alternative branches and could not generalize to novel player behaviours. Riedl and Bulitko (2013) provided a comprehensive survey of intelligent interactive narrative systems, identifying the tension between authorial control and player agency as the central design challenge.

More recent work has approached the problem from a content generation perspective. Procedural narrative generation systems, such as Tracery (Compton et al., 2015) and the Expressionist framework (Kreminski et al., 2019), demonstrated that grammatical expansion could produce contextually varied textual content. However, such systems still required extensive authoring of production rules and lacked the capacity to respond to the player's psychological state. Short (2019) argued that procedural generation without a model of player intent risks producing content that is varied but meaningless — quantity of output without quality of relevance.

The commercial industry has converged on a middle path: narrative choice systems with authored branches (e.g., *Mass Effect*, *The Witcher* series, *Disco Elysium*) provide the illusion of personalization through meaningful choices but are ultimately deterministic in their narrative delivery relative to choice history. These systems do not adapt to behavioural signals such as play pace, combat avoidance, or exploration rate — they adapt only to explicit player-authored choices within a pre-specified decision space. The cost of authoring these branches is substantial: *Disco Elysium*, widely regarded as one of the most narratively ambitious RPGs, contains approximately one million words of authored text, yet still delivers a fixed narrative structure relative to the player's explicit choices.

The emerging paradigm, exemplified by PANGeA (Todd et al., 2024), uses generative AI to produce narrative content at runtime, opening the possibility of adaptation that is both infinite in variety and grounded in the game's content universe. The current work situates itself at the intersection of this generative paradigm and player modelling: the narrative is not merely generated but selected and styled in response to a continuous model of the individual player's state.

2.3 Player Modelling Approaches

2.3.1 Bartle Taxonomy and Its Limitations

Bartle's (1996) taxonomy of MUD player types — Achievers, Explorers, Socializers, and Killers — was the first systematic attempt to categorize players by behavioural motivation. Developed through observation of Multi-User Dungeon players in the 1980s, the taxonomy introduced the foundational insight that players seek qualitatively different experiences from the same game.

Despite its wide adoption in game design pedagogy, the Bartle taxonomy has several well-documented limitations. First, it is categorical rather than continuous: a player is assigned to one type, when empirical evidence suggests players express multiple motivations simultaneously and in varying proportions (Yee, 2006). Second, the taxonomy was derived from a narrow population playing a specific genre (text-based MUDs) and has questionable generalizability to modern games. Third, the categories are not mutually exclusive and lack operationalized measurement criteria. Finally, type assignment relies on self-report questionnaires (the Bartle Test), which introduce social desirability bias and are one-time snapshots rather than dynamic, within-session measures.

2.3.2 Hexad Typology

Tondello et al. (2016) developed the Gamification User Types Hexad Scale as a theoretically grounded alternative to Bartle's taxonomy. Rooted in Self-Determination Theory (Ryan and Deci, 2000), the Hexad framework identifies six player types: Achievers (motivated by mastery and progress), Explorers (motivated by discovery and curiosity), Socializers (motivated by social interaction and belonging), Free Spirits (motivated by autonomy and self-expression), Disruptors (motivated by challenging the status quo), and Philanthropists (motivated by giving and altruism). Unlike Bartle's types, Hexad scores are continuous (typically measured on a seven-point Likert scale per item) and can be meaningfully combined.

The original Hexad scale is a 24-item self-report questionnaire, validated for gamification contexts with acceptable reliability (Cronbach's alpha typically exceeding 0.70 per subscale). Subsequent work has applied Hexad profiling to adaptive game design, recommendation systems, and educational technology (Hamari et al., 2014), establishing it as the de facto standard for research-grade player-type assessment.

The critical limitation for the current work is the reliance on a pre-play questionnaire. A questionnaire-derived profile is a static snapshot, captured before any gameplay, that cannot update as the player's engagement patterns evolve during a session. Moreover, administering a 24-item questionnaire before play introduces friction that reduces ecological validity. The present work addresses this limitation by inferring Hexad-analogous continuous scores directly from behavioural telemetry — a novel contribution detailed in Section 3.3.

2.3.3 Data-Driven Player Modelling

The player2vec line of research (2024) demonstrates the application of Transformer-based self-supervised learning to player behaviour sequences. Analogous to word2vec embeddings for natural language tokens, player2vec encodes sequences of in-game actions into dense low-dimensional representations that cluster players by behavioural similarity without requiring labelled training data. Zhu et al. (2023) provided a comprehensive survey of player modelling approaches for interactive narrative, identifying the shift from questionnaire-based to behaviour-based modelling as a key trend.

The current work draws conceptual inspiration from player2vec — specifically, the notion that a session embedding can summarize a player's behavioural trajectory — but implements a simpler rule-based approach rather than a trained Transformer. This pragmatic choice is motivated by the cold-start constraint: a Transformer session embedder requires a substantial corpus of historical play sequences to produce meaningful representations, which is unavailable for a first-session player using a newly deployed plugin.

2.4 Flow Theory in Game Design

Csikszentmihalyi's (1990) theory of Flow describes a psychological state of optimal experience characterized by complete absorption in a challenging activity, loss of self-consciousness, and distorted time perception. Flow arises at the intersection of high challenge and high skill: when challenge substantially exceeds skill, the result is anxiety; when skill substantially exceeds challenge, the result is boredom; and when both are absent, the result is apathy. Nakamura and Csikszentmihalyi (2002) subsequently refined the model, emphasizing that Flow is not a binary state but a continuum influenced by multiple contextual factors.

In game design, Flow theory has been influential since Chen's (2007) canonical analysis of game flow, which argued that the dynamic difficulty adjustment (DDA) mechanisms present in many games (e.g., adaptive enemy scaling, hint systems) are implicitly Flow-optimization mechanisms. The challenge–skill ratio has since become a standard construct for computational Flow modelling.

The present work operationalizes Csikszentmihalyi's model through a rule-based classifier that maps the `challenge_skill_ratio` (computed from behavioural telemetry) to one of four states: FLOW (ratio in [0.75, 1.30]), ANXIETY (ratio > 1.30), BOREDOM (ratio < 0.75), and APATHY (a joint condition of high idle time, low dialogue engagement, and low challenge–skill ratio). These thresholds are informed by prior work on computational Flow detection (Yannakakis and Hallam, 2009) and are detailed formally in Section 3.3.4. Critically, the reward function of the PPO adaptation engine is directly shaped around these states, making Flow the central organizing principle of the adaptation objective.

2.5 Large Language Models for Game Narrative

2.5.1 PANGeA (AAAI AIIDE 2024)

Todd et al.'s (2024) Procedural Artificial Narrative using Generative AI (PANGeA) represents the most directly relevant antecedent work for the narrative generation component of the current system. PANGeA employs an LLM to generate narrative content procedurally, grounded in a structured game knowledge base delivered through the prompt. The key insight from PANGeA is that LLMs can produce coherent, in-world narrative content when given sufficient contextual grounding — but require careful prompt engineering and content validation to avoid outputs that break the fictional frame.

PANGeA does not, however, integrate a real-time player model or an adaptive policy. Its narrative generation is procedural (driven by game state) rather than personalized (driven by player psychological state). The current work extends the PANGeA paradigm by adding the player modelling and RL adaptation layers that PANGeA lacks.

2.5.2 LIGS (CHI 2025)

The LLM-Integrated Game Storytelling (LIGS) system, published at CHI 2025, investigated the integration of LLMs into game narrative pipelines with a focus on maintaining narrative coherence across extended play sessions. LIGS introduced mechanisms for tracking narrative state across sessions and using that state to condition generation, anticipating the episodic memory approach of the current work.

A notable contribution of LIGS was its evaluation methodology: it employed player experience questionnaires to compare AI-generated narrative against human-authored content, finding that players could not reliably distinguish the two when the LLM was appropriately constrained. This finding provides empirical support for the feasibility of LLM-based narrative personalization as a subjectively valid approach.

2.5.3 Prompt Grounding and Retrieval-Augmented Generation

The challenge of keeping LLM outputs factually and fictionally consistent with a specific game world has motivated the adoption of Retrieval-Augmented Generation (RAG; Lewis et al., 2020) in game narrative contexts. RAG supplements the LLM prompt with retrieved passages from a curated knowledge base, effectively providing the model with relevant context at generation time rather than requiring this knowledge to be baked into model weights through fine-tuning.

For game narrative, RAG offers a particularly attractive trade-off: the lore corpus can be maintained and updated independently of the model, and retrieval ensures that only contextually relevant lore is included in the prompt, avoiding context window saturation. The Transformer-based architecture underlying modern LLMs (Vaswani et al., 2017) processes fixed-size context windows, making efficient context usage critical for generation quality. The current system employs the sentence-transformers all-MiniLM-L6-v2 model (Wang et al., 2020) to embed both the lore corpus and generation queries into a 384-dimensional semantic space, with top-k cosine similarity retrieval ($k = 3$) providing the relevant passages.

2.6 Reinforcement Learning for Game Adaptation

2.6.1 Dynamic Difficulty Adjustment

Dynamic Difficulty Adjustment (DDA) has been an active research area since Hunicke and Chapman (2004) demonstrated that adaptive difficulty improved player retention in *Super Mario Bros*. Computational approaches to DDA have employed rule-based systems, Bayesian models, and neural networks to infer the appropriate challenge level from behavioural signals and adjust game parameters accordingly (Lopes and Bidarra, 2011). The predominant formulation is as a feedback control problem: the challenge level is a controlled variable, player engagement state is observed, and the controller minimizes the deviation from a target engagement state.

The present work extends the DDA paradigm in two important respects. First, the adaptation target is not mechanical difficulty (enemy health, spawn rate, puzzle complexity) but *narrative content*: the tone, urgency, and type of narrative delivery. Second, the adaptation mechanism is a learned policy rather than a hand-crafted controller. These extensions transform DDA from a mechanical tuning problem into a narrative and psychological design problem.

2.6.2 PPO and Stable-Baselines3

Proximal Policy Optimization (PPO; Schulman et al., 2017) is a policy gradient reinforcement learning algorithm that achieves reliable performance across a broad range of environments through a clipped surrogate objective that prevents destructively large policy updates. PPO has become a standard baseline in both academic RL research and practical game-playing applications, owing to its relative simplicity, sample efficiency, and stability compared to earlier policy gradient methods (Mnih et al., 2015).

Stable-Baselines3 (Raffin et al., 2021) provides production-quality PyTorch implementations of PPO and other RL algorithms, with interfaces compatible with the Gymnasium (formerly OpenAI Gym; Brockman et al., 2016) environment specification. The current work employs Stable-Baselines3 v2.x with the MlpPolicy architecture, training over 200,000 timesteps in a custom Gymnasium environment that simulates narrative adaptation episodes. The use of Stable-Baselines3 ensures reproducibility and provides a well-tested optimization implementation.

2.7 Episodic Memory for LLMs

Fountas et al. (2024) introduced EM-LLM, an architecture for endowing LLMs with human-like episodic memory. The system partitions a long context stream into discrete memory events, scores them by importance and surprise, and selectively retrieves the most relevant events for inclusion in future prompts. This approach enables a form of infinite context through principled compression: rather than attending to every prior interaction, the model recalls the events most salient to the current context.

The current work incorporates a lightweight episodic memory inspired by EM-LLM. The `EpisodicMemory` class maintains up to ten narrative events, each scored by importance, and evicts the least important event when capacity is exceeded. At generation time, the top five events by importance score are retrieved and formatted as a context block in the LLM prompt. This ensures narrative continuity — the generated content acknowledges prior story events — without saturating the context window. The simplification from EM-LLM's full architecture is appropriate given the relatively short session duration (30 minutes) and the modest number of narrative events generated per session (approximately 15–20).

2.8 Research Gap and Positioning

Table 2.1 summarizes the key related works and their coverage of the dimensions relevant to this study.

Table 2.1: Summary of related work on narrative personalization and player modelling.

Work	Player Model	Flow Reward	LLM Gen.	RAG	Episodic Memory	Complete Loop
PANGeA (Todd et al., 2024)	No	No	Yes	Partial	No	No
LIGS (CHI 2025)	No	No	Yes	No	Partial	No
DDA (Hunicke and Chapman, 2004)	Behav.	Implicit	No	No	No	No
player2vec (2024)	Deep	No	No	No	No	No
EM-LLM (Fountas et al., 2024)	No	No	Yes	No	Yes	No
This work	**Hexad+ Flow**	**Explicit**	**Yes (RAG)**	**Yes**	**Yes**	**Yes**

No prior work has integrated continuous Hexad profiling derived from telemetry, Flow-based RL reward, RAG-grounded LLM generation, and episodic session memory into a single closed-loop system. The present work addresses this gap by combining all six components into a deployable Unity plugin, evaluated through a rigorous mixed-methods study design.

Chapter 3

METHODOLOGY

3.1 Chapter Overview

This chapter presents the research methodology in detail. It begins with the overall research design and philosophical positioning, then systematically describes each of the five system modules: telemetry collection, player modelling, narrative generation, RL-based adaptation, and Unity plugin design. The chapter then describes the testbed game environment and concludes with the full evaluation design, including hypotheses, instruments, procedure, ethical considerations, and analysis plan. The methodology follows a mixed-methods approach (Creswell and Plano Clark, 2018), combining a controlled between-subjects experiment with qualitative interview analysis.

3.2 Research Design Overview

The research adopts a mixed-methods design combining a controlled between-subjects experiment with a qualitative interview study. The experimental component enables causal inference about the effect of the AI-driven personalization system on player experience, while the qualitative component captures the lived experience of personalization that quantitative measures may not fully represent.

The between-subjects design was chosen over a within-subjects design to eliminate order effects and carry-over: a player who first experiences adaptive narrative and then static narrative (or vice versa) would bring expectations and memory from the first condition into the second. With a between-subjects design, each participant experiences only one condition, removing this contamination at the cost of requiring a larger sample and careful matching.

The experimental condition (Experimental) deploys the full five-module AI system. The control condition (Control) replaces the Narrative Generation Service and Adaptation Engine with a `StaticNarrativeContent` ScriptableObject that delivers pre-authored narrative content in response to the same game triggers, with no player-adaptive behaviour. All other aspects of the game session — environment, objectives, NPC placement, session duration — are identical across conditions. This design isolates the effect of the adaptive narrative system from confounding factors related to game design or content quality differences.

3.3 System Architecture

The system is organized as five modules in a closed-loop pipeline. Figure 3.1 describes the architecture and the data flows between components.

[Insert system architecture diagram here — five-module closed-loop pipeline showing: Unity Client (PlayerDataLogger) → FastAPI Backend (/telemetry/batch) → Player Modelling Service → Adaptation Engine (PPO) → Narrative Generation Service (RAG + LLM) → Unity Client (NarrativeManager + ContentInjector) → Player → Unity Client]

Figure 3.1: High-level closed-loop architecture showing the five-module pipeline and data flows between components.

The pipeline operates as follows. The Unity `PlayerDataLogger` aggregates behavioural signals over five-second windows and transmits a `TelemetryBatch` to the FastAPI backend via HTTP POST (with WebSocket support for lower-latency streaming). The Player Modelling Service processes incoming batches: the `FeatureExtractor` computes eleven normalized features, the `HexadProfiler` derives the six-dimensional continuous Hexad profile, and the `FlowClassifier` assigns a Flow state. The resulting `PlayerModel` is passed to the `NarrativeAgent`, which selects a `NarrativeAction` using either the cold-start heuristic (first 120 seconds) or the trained PPO policy. The Narrative Generation Service uses the selected action to query the `LoreRetriever` (RAG), consults the `EpisodicMemory`, assembles the full prompt through the `PromptBuilder`, and requests generation from the Ollama-hosted LLM. The resulting `NarrativeContent` is returned to the Unity client and injected at the appropriate in-game display point.

3.4 Player Modelling Module

3.4.1 Telemetry Collection

The telemetry system captures 23 behavioural fields per five-second window. Table 3.1 lists the complete schema.

Table 3.1: TelemetryBatch schema (23 fields).

Field	Type	Description
player_id	str	Unique player identifier
session_id	str	Unique session identifier
window_start	float	Window start time (Unix timestamp)
window_end	float	Window end time (Unix timestamp)
kills	int	Enemy kills in window
deaths	int	Player deaths in window
damage_taken	float	Total damage received
damage_dealt	float	Total damage inflicted
abilities_used	int	Ability activations
abilities_hit	int	Abilities that connected
objectives_completed	int	Objectives completed
objectives_attempted	int	Objectives attempted
tiles_explored	int	Map tiles visited
total_tiles	int	Total traversable tiles (default 100)
npc_interactions_voluntary	int	Voluntary (unsolicited) NPC interactions
dialogue_lines_shown	int	Dialogue lines displayed
dialogue_lines_skipped	int	Dialogue lines skipped by player
lore_items_read	int	Lore collectibles read
lore_items_found	int	Lore collectibles discovered
idle_seconds	float	Seconds with no player input
session_elapsed	float	Total elapsed session time (seconds)
current_location	str	Scene/zone identifier
current_objective	str	Active quest objective identifier

The five-second window granularity balances responsiveness (enabling sub-minute adaptation cycles) against noise sensitivity (single-frame events are smoothed over the window). In practice, the system accumulates a sliding window of recent batches before computing the feature vector, allowing multi-window averages to further smooth transient spikes.

3.4.2 Feature Extraction (11 Features)

The `FeatureExtractor` aggregates one or more `TelemetryBatch` records into eleven normalized scalar features, each clipped to [0, 1] (or [0, 3] for the `challenge_skill_ratio`). Table 3.2 presents the computation formulae.

Table 3.2: Eleven extracted features and computation formulae.

Feature	Formula	Notes
kd_ratio	clip(kills / max(deaths, 1) / 5.0, 0, 1)	Norm. at 5:1 K/D
objective_rate	clip(obj_comp / max(obj_att, 1), 0, 1)	Completion proportion
explore_rate	clip(tiles_explored / total_tiles, 0, 1)	Map coverage
dialogue_engage	clip(1 − skipped / max(shown, 1), 0, 1)	1 = reads all
ability_accuracy	clip(hit / max(used, 1), 0, 1)	Combat precision
damage_taken_norm	clip(dmg / (elapsed_min + 1) / 200, 0, 1)	DPM normalized
idle_ratio	clip(idle_s / elapsed, 0, 1)	Fraction idle
lore_engage_rate	clip(read / max(found, 1), 0, 1)	Lore depth
skill_score	0.35 kd + 0.30 obj + 0.20 acc + 0.15(1−idle)	Composite skill
challenge_score	0.50 dmg + 0.30(1−obj) + 0.20 death_ratio	Composite challenge
challenge_skill_ratio	clip(challenge / max(skill, 10^{-6}), 0, 3)	Core Flow signal

The `skill_score` composite weights combat effectiveness (`kd_ratio`, `ability_accuracy`) most heavily, followed by objective completion and attention (inverse `idle_ratio`). The `challenge_score` composite weights damage pressure most heavily, followed by objective failure rate and death rate. Dividing challenge by skill yields the `challenge_skill_ratio`, which operationalizes Csikszentmihalyi's (1990) challenge–skill balance axis.

3.4.3 Hexad Profile Computation (6 Dimensions)

The `HexadProfiler` computes six continuous scores in [0, 1] from accumulated telemetry, each representing the strength of a Hexad player-type signal. Table 3.3 shows the computation weights.

Table 3.3: Hexad dimension computation weights from telemetry signals.

Dimension	Formula	Interpretation
Achiever	0.70 × obj_rate + 0.30 × norm(kills, 20)	Objective + combat
Explorer	0.50 × explore_rate + 0.50 × lore_rate	Map + lore coverage
Socializer	0.60 × norm(vol_npc, 10) + 0.40 × dialogue	Seeks NPCs + reads
Free Spirit	0.50 × explore_rate + 0.50 × (1 − obj_rate)	Explores off-path
Disruptor	0.65 × norm(kills, 20) + 0.15 × norm(deaths, 10) + 0.20 × norm(dmg, 1000)	Aggressive combat
Philanthropist	0.60 × dialogue_engage + 0.40 × lore_rate	Narrative engagement

The mapping from behavioural signals to Hexad dimensions was established through a principled alignment process guided by the original Hexad motivation descriptions (Tondello et al., 2016): Achievers seek mastery and progress (captured by objective completion and combat effectiveness); Explorers seek discovery (captured by map and lore coverage); Socializers seek connection (captured by voluntary NPC interaction and dialogue engagement); Free Spirits resist imposed structure (captured by exploration uncorrelated with objective completion); Disruptors challenge systems (captured by aggressive combat behaviour); Philanthropists give and share (captured by engagement with narrative and lore content).

All profiles are initialized to 0.5 (neutral) in the absence of data, preventing cold-start artefacts from biasing early narrative selection.

3.4.4 Flow State Classification (MDP States)

The `FlowClassifier` implements a decision-tree classifier over the `challenge_skill_ratio`, `idle_ratio`, and `dialogue_engage_rate` features. Table 3.4 formalizes the rules.

Table 3.4: Flow state classification rules (evaluated in priority order).

Priority	Condition	State	Confidence
1 (highest)	idle $>$ 0.40 AND dialogue $<$ 0.30 AND ratio $<$ 0.60	APATHY	0.70
2	ratio $<$ 0.75	BOREDOM	0.60 + (0.75 $-$ ratio) $\times$ 0.80
3	ratio $>$ 1.30	ANXIETY	0.60 + (ratio $-$ 1.30) $\times$ 0.40
4 (default)	0.75 $\leq$ ratio $\leq$ 1.30	FLOW	max(0.55, 1.0 $-$ $\lvert$ratio $-$ 1.0$\rvert$ $\times$ 1.2

APATHY is classified first because it represents a qualitatively distinct disengagement pattern (high inactivity + low narrative interaction + low challenge) that would otherwise be misclassified as BOREDOM on the ratio criterion alone. The confidence scoring for BOREDOM and ANXIETY is proportional to the distance from the FLOW band boundary, reflecting that states further from the boundary are classified with higher certainty. FLOW confidence is highest when the ratio is closest to 1.0 (perfect challenge–skill balance).

3.5 Narrative Generation Module

3.5.1 RAG Pipeline

The `LoreRetriever` implements in-memory semantic search using the sentence-transformers all-MiniLM-L6-v2 model, which produces 384-dimensional L2-normalized embeddings. The lore corpus is pre-indexed at startup by chunking the world, character, and quest lore documents by paragraph (double-newline delimiters) and embedding each chunk. At retrieval time, the query is embedded and cosine similarities are computed against all stored embeddings through a dot product (since all vectors are unit-normalized, cosine similarity equals the dot product).

The top $k = 3$ chunks by similarity are returned and formatted as bullet points in the LLM user prompt.

This design was chosen over a persistent vector database (e.g., ChromaDB, Pinecone) for two reasons: Python 3.14 compatibility issues with ChromaDB's Pydantic v1 dependency, and the relatively small lore corpus size (three files, approximately 30 paragraphs total) which renders the overhead of a full vector database unnecessary. In-memory retrieval at this corpus size is effectively instantaneous.

3.5.2 Episodic Session Memory

The `EpisodicMemory` class maintains a capped list of `MemoryEvent` records, each comprising a text description, an importance score in [0, 1], and a timestamp. When the list exceeds the maximum capacity of ten events, it is sorted by importance in descending order and truncated to the top ten. This importance-weighted eviction ensures that the most narratively significant events are retained even as minor events accumulate.

At prompt construction time, the top five events by importance score are selected and formatted as a context block, providing the LLM with a compressed narrative history that enables references to past events and prevents temporal inconsistency.

3.5.3 Dynamic Prompt Construction

The `PromptBuilder` assembles a two-message prompt (system + user) for the Ollama API. The system prompt encodes the LLM's role (narrative writer for the game world), output length constraints (2–4 sentences for dialogue, 1–2 for descriptions), format requirements (valid JSON only, specific schema), and the emotional tone mapped from the current Flow state:

- ANXIETY: "urgent and supportive — the player is struggling"
- BOREDOM: "exciting and surprising — inject energy"
- FLOW: "immersive and atmospheric — maintain the experience"
- APATHY: "intriguing and mysterious — re-engage the player"

The user prompt injects: the player's current Flow state and challenge–skill ratio; the current activity (IN_COMBAT, EXPLORING, IN_DIALOGUE, IDLE); session elapsed time; the Hexad profile percentages; the selected NarrativeAction with definitions; the current location and active quest stage; the three retrieved lore chunks; the episodic memory context; and explicit content constraints (no real-world references, no anachronisms, no fourth-wall breaks, character voice descriptions).

3.5.4 LLM Integration (Ollama / Phi-3.5 Mini)

The `OllamaClient` sends the assembled prompt to a locally hosted Ollama instance using the `/api/chat` endpoint. The primary model is Phi-3.5 Mini (3.8B parameters; Microsoft, 2024), with Llama 3.2 3B as a configured fallback. The generation parameters are: temperature 0.75, top_p 0.90, maximum tokens 200, JSON format mode enabled. The 8-second HTTP timeout ensures that a slow or stalled generation does not block the main game loop.

When the LLM response is malformed JSON, contains a fallback signal, or the request times out, the `OllamaClient` returns a pre-authored fallback string corresponding to the requested `NarrativeAction`. These fallback strings are minimal, tonally appropriate responses that maintain narrative coherence without exposing the player to system failure artefacts.

3.6 Adaptation Engine (MDP + PPO)

3.6.1 MDP Formulation

The narrative adaptation problem is formalized as a Markov Decision Process (S, A, T, R, γ) implemented as a Gymnasium environment.

State space S: A Box observation space of shape (7,) with all values in [0, 1]. Table 3.5 defines the seven dimensions.

Table 3.5: MDP observation vector (7 dimensions).

Index	Signal	Description
0	flow_score	Encoded Flow state (FLOW=1.0, BOREDOM=0.3, ANXIETY=0.2, APATHY=0.1)
1	csr_norm	challenge_skill_ratio / 3.0
2	hexad_explorer	Explorer dimension score
3	hexad_socializer	Socializer dimension score
4	hexad_achiever	Achiever dimension score
5	hexad_disruptor	Disruptor dimension score
6	session_progress	session_elapsed / 1800.0 (normalized to 30-min max)

Action space A: A Discrete(8) action space corresponding to the eight `NarrativeAction` values. Table 3.6 summarizes the actions.

Table 3.6: Eight narrative actions with descriptions and intended effects.

Idx	Action	Intended Effect
0	LOWER_STAKES	Reduce narrative tension; provide respite
1	RAISE_STAKES	Increase narrative urgency and weight
2	ADD_MYSTERY	Introduce a curiosity hook or unexplained element
3	ADD_HUMOR	Provide comic relief to reduce tension
4	PROVIDE_GUIDANCE	Give struggling player directional narrative support
5	INCREASE_URGENCY	Re-energize a bored or disengaged player
6	LORE_REWARD	Deliver a lore discovery payoff for curious players
7	NO_CHANGE	Suppress narrative injection; maintain current experience

Transition dynamics T: The simulated training environment models stochastic Flow state transitions. When the agent selects a "helpful" action for the current state (e.g., ANXIETY $\rightarrow$ {PROVIDE_GUIDANCE, LOWER_STAKES}), the environment transitions to FLOW with probability 0.75 or remains in the current state with probability 0.25. This stochasticity prevents the policy from memorizing a deterministic mapping and encourages robust adaptation.

Discount factor γ: 0.95, reflecting moderate preference for near-term reward while maintaining long-horizon planning.

3.6.2 Reward Function Design

The reward function is defined over (prev_state, new_state, action, steps_in_state, last_3_actions) and comprises five components, detailed in Table 3.7.

Table 3.7: Reward function components and values.

Component	Value	Trigger
State reward (FLOW)	$+1.0$	new_state == FLOW
State reward (BOREDOM)	-0.3	new_state == BOREDOM
State reward (ANXIETY)	-0.5	new_state == ANXIETY
State reward (APATHY)	-0.8	new_state == APATHY
Step penalty	-0.05	Every step
Flow streak bonus	$+0.20$	FLOW AND steps_in_state > 2
Transition bonus	$+0.30$	prev $\in$ {ANXIETY, BOREDOM} AND new == FLOW
Repetition penalty	-0.15	All 3 recent actions identical

The state rewards reflect the severity ordering of non-Flow states: APATHY is penalized most heavily because it represents the most concerning disengagement. The step penalty provides constant pressure toward action efficiency. The Flow streak bonus reinforces sustained optimal experience. The transition bonus provides additional shaping reward for key recovery transitions. The repetition penalty discourages degenerate policies.

3.6.3 Cold-Start Heuristic

For the first 120 seconds of a session, or when the PPO model is unavailable, the `NarrativeAgent` falls back to a deterministic heuristic policy:

- ANXIETY → PROVIDE_GUIDANCE
- BOREDOM → INCREASE_URGENCY
- APATHY → ADD_MYSTERY
- FLOW → NO_CHANGE

The 120-second threshold allows the player sufficient time to engage with the game world before the RL policy takes effect, giving the player modelling module time to accumulate meaningful telemetry.

3.7 Unity Plugin Design

The Unity plugin comprises three C# components: `PlayerDataLogger`, `NarrativeManager`, and `ContentInjector`.

`PlayerDataLogger` is a MonoBehaviour that maintains running accumulators for all 23 telemetry fields. Every five seconds, it serializes the accumulated values into a `TelemetryBatch` JSON object and dispatches it to the FastAPI backend via an async HTTP POST. It exposes a public API for game systems to register events: `RecordKill()`, `RecordDeath()`, `RecordDialogueShown()`, and others. This event-based registration model decouples the telemetry system from specific game mechanics, making the plugin applicable to any Unity game that exposes these event hooks.

`NarrativeManager` polls the `/narrative/generate` endpoint following each telemetry transmission. `ContentInjector` maps `NarrativeContentType` to the appropriate UI mechanism: dialogue content is routed to the DialoguePanel, description content to the DescriptionPanel, and quest updates to the QuestLog. The injector implements a content queue with a 3-second minimum spacing between consecutive injections to prevent narrative overcrowding.

For the Control condition, the `NarrativeManager` is replaced by a `StaticNarrativeManager` that reads from a `StaticNarrativeContent` ScriptableObject and fires narrative events at scripted triggers with no player-state dependency.

3.8 Testbed Game Design

The testbed is a 2D top-down RPG built in Unity 6 LTS. The game world encompasses a warzone setting — the Contested Vale — a once-fertile trade corridor now fought over by two factions. The world is described in structured lore documents that serve as the RAG corpus.

The game world features three navigable areas:

1. **Commander Varis's Post** — a fortified camp at the northern edge serving as the starting area. Varis is the primary quest giver: a methodical, unsentimental military commander who gives orders without explaining them. Players who engage extensively with Varis register high Achiever signals.

2. **The Contested Zone** — a combat-focused area containing enemy units and environmental hazards. Players who clear this area efficiently register high Achiever and Disruptor signals; players who explore its periphery register Explorer signals.

3. **The Waystone / Chronicler's Camp** — a neutral area where the Chronicler, a scholar who has observed the Vale for thirty years, provides lore and context. The presence of lore collectibles creates a natural Explorer/Philanthropist signal divergence.

Three NPCs populate the world: Commander Varis (primary quest giver, military voice), the Chronicler (lore source, academic voice), and Sera the Merchant (supply trader, pragmatic voice). The main quest (Clear the Advance) guides players through the combat zone, while a collection quest (Salvage the Fallen) encourages exploration. A standard play session is approximately 25–30 minutes.

3.9 Evaluation Design

3.9.1 Research Hypotheses

The following directional hypotheses are tested:

H1: Participants in the Experimental condition will score significantly higher on the GEQ Flow subscale than participants in the Control condition.

H2: Participants in the Experimental condition will score significantly higher on the GEQ Immersion subscale than participants in the Control condition.

H3: Participants in the Experimental condition will score significantly higher on the NPS-3 Narrative Personalization Scale than participants in the Control condition.

H1 is designated the primary hypothesis because Flow is the direct optimization target of the adaptation engine.

3.9.2 Participants and Recruitment

A total of $N = 50$ participants are recruited (25 Experimental, 25 Control) via convenience sampling through university mailing lists and gaming society channels. Inclusion criteria: aged 18 or over; self-reported experience playing RPG or action-adventure games; no prior exposure to the testbed game. An a priori power analysis for Welch's independent-samples t-test targeting $d = 0.5$ (medium effect), $\alpha = 0.05$ (two-tailed), and $1 - \beta = 0.80$ yields a required sample size of approximately 51 total participants (Faul et al., 2007), indicating that $N = 50$ is marginally below the strict threshold but remains reasonable given practical constraints.

Participants are randomly assigned to conditions using a balanced random allocation procedure. No block randomization by demographic is applied, given the expected homogeneity of the university convenience sample.

3.9.3 Instruments

Game Experience Questionnaire (GEQ) — Core Module. The GEQ (IJsselsteijn et al., 2013) is a 33-item instrument measuring seven dimensions of game experience on a 0–4 scale. The primary DV is the GEQ **Flow subscale** (items 5, 13, 25, 28, 31; items 5 and 13 reverse-scored). The secondary DV is the GEQ **Immersion subscale** (items 3, 12, 18, 19, 27, 30).

miniPXI — Player Experience Inventory (Short Form). The miniPXI (Vornhagen et al., 2022) is an 11-item instrument on a 1–7 scale covering six player experience dimensions. The total score (mean of all 11 items) is used as a holistic player experience index.

Narrative Personalization Scale (NPS-3). The NPS-3 is a custom three-item scale on a 1–7 Likert scale: (1) "The story felt tailored to how I was playing," (2) "The narrative responded to my choices and actions," and (3) "The game seemed to understand what kind of player I am." Cronbach's alpha will be reported; if $\alpha < 0.70$, items will be analysed individually.

Behavioural Metrics (from telemetry). Three behavioural metrics derived from telemetry serve as objective secondary measures: voluntary NPC interactions, dialogue read ratio, and exploration coverage.

3.9.4 Procedure

Each participant session follows a standardized procedure conducted individually:

1. **Arrival and consent** (5 min): briefing, participant information sheet, informed consent, audio recording consent.

2. **Tutorial** (5 min): condition-identical tutorial scene familiarizing participants with movement, combat, and NPC interaction.

3. **Game session** (25–30 min): participant plays in their assigned condition. The researcher is present but silent. Session is time-limited to 30 minutes.

4. **Questionnaires** (10 min): GEQ Core Module, miniPXI, and NPS-3 completed immediately post-session.

5. **Semi-structured interview** (20 min): following the interview guide (Appendix D), audio recorded.

6. **Debrief** (5 min): explanation of the system and opportunity for questions.

Total session duration: approximately 65 minutes per participant.

3.9.5 Ethical Considerations

This research received ethical approval through the institutional SPER process. All participants provide informed consent prior to participation and are informed of their right to withdraw at any time without penalty. Telemetry data is stored locally on the research machine in a SQLite database without personally identifiable information beyond the session identifier. Audio recordings from interviews are stored on an encrypted drive and transcribed with participant consent. All data is handled in accordance with GDPR requirements and university data protection policy. Participants are debriefed following the session and informed of the study's full purpose.

3.9.6 Analysis Plan

Quantitative analysis follows these steps:

1. **Normality assessment:** Shapiro-Wilk tests per DV per condition. If normality is violated ($p < 0.05$), Mann-Whitney U is substituted for Welch's t-test.

2. **Primary test (H1):** Welch's independent-samples t-test comparing GEQ Flow subscale scores. Effect size reported as Cohen's d (Cohen, 1988).

3. **Secondary tests (H2, H3):** Identical procedure. Bonferroni correction applied across three hypotheses ($\alpha_{\text{adj}} = 0.0167$).

4. **Reliability:** Cronbach's alpha computed for each subscale.

5. **Behavioural metrics:** Welch's t-tests as exploratory analyses without correction.

Qualitative analysis follows the six-phase Reflexive Thematic Analysis procedure (Braun and Clarke, 2022): familiarization, coding, constructing themes, reviewing, defining themes, and writing up with representative quotes.

Chapter 4

SYSTEM IMPLEMENTATION

4.1 Chapter Overview

This chapter documents the technical implementation of the five-module system described in Chapter 3. It details the technology stack, backend service architecture, database design, player modelling implementation, narrative pipeline, RL adaptation engine, and Unity plugin. The chapter also describes the testing and validation procedures applied to ensure system correctness.

4.2 Technology Stack

The system is implemented across two platforms: a Python backend service and a Unity C# plugin. Table 4.1 and Table 4.2 summarize the key dependencies.

Table 4.1: Backend technology stack.

Component	**Technology**	**Version**
Runtime	Python	3.14
API framework	FastAPI	0.115+
Data validation	Pydantic	v2
Database	SQLite	(stdlib)
RL framework	Stable-Baselines3	2.x
RL environment	Gymnasium	0.29+
Embeddings	sentence-transformers	latest
LLM client	httpx (Ollama API)	0.27+
Numerical	NumPy	2.x
Statistics	SciPy, pingouin	latest
Data analysis	pandas	2.x

Table 4.2: Unity plugin and inference stack.

Component	Technology	Version
Engine	Unity	6 LTS
Language	C#	.NET 9
HTTP client	UnityWebRequest	built-in
JSON	Newtonsoft.Json	13.x
UI	TextMeshPro	built-in
LLM runtime	Ollama (local)	—
Primary model	Phi-3.5 Mini	3.8B params
Fallback model	Llama 3.2 3B	—

4.3 Backend Service

The FastAPI backend exposes four primary endpoints:

- `POST /telemetry/batch` — accepts a `TelemetryBatch` payload, persists it to SQLite, triggers player modelling, and initiates the adaptation cycle.
- `GET /narrative/generate` — returns the latest `NarrativeContent` for a given session, blocking until generation is complete.
- `GET /player/model/{player_id}` — returns the current `PlayerModel` including Hexad profile, Flow state, and session metadata.
- `WebSocket /ws/telemetry/{session_id}` — low-latency telemetry streaming channel.

The server is configured with CORS middleware to accept connections from the Unity client's localhost origin. All data is persisted to SQLite with separate tables for telemetry batches, player models, and narrative events. Pydantic v2 models handle all input validation and output serialization, ensuring type safety at the API boundary.

The asynchronous architecture of FastAPI allows the narrative generation pipeline — including the 8-second Ollama timeout — to execute without blocking other incoming telemetry requests. This is critical for maintaining the 5-second telemetry cycle even when LLM generation is in progress.

4.4 Database Layer

The SQLite schema uses three primary tables:

1. **telemetry_batches** — stores all 23 `TelemetryBatch` fields plus a server-side `received_at` timestamp. Indexed on `(player_id, session_id, window_start)`.

2. **player_models** — stores `PlayerModel` snapshots indexed on `(player_id, session_id, timestamp)`. Each telemetry POST triggers an upsert of the computed model.

3. **narrative_events** — stores `NarrativeContent` records (action, content_type, content, speaker, emotional_tone, fallback flag) with foreign keys to session_id. This log enables post-hoc analysis of which narrative actions were fired at what points in each session.

SQLite was selected over PostgreSQL for simplicity of deployment (no external service required), study scale ($N = 50$ generates modest data volumes), and the single-writer access pattern of the study setup.

4.5 Player Modelling Implementation

The player modelling pipeline is implemented as three sequential classes called within the same request handler:

Listing 4.1: Player modelling pipeline flow.

```
TelemetryBatch[] (from SQLite)
  -> FeatureExtractor.extract()  -> dict[str, float] (11 features)
  -> HexadProfiler.compute()     -> HexadProfile      (6 dims)
  -> FlowClassifier.classify()   -> (FlowState, float) (state +
     conf)
  -> PlayerModel (assembled)
```

The `FeatureExtractor` aggregates all `TelemetryBatch` records for the current session from SQLite before computing features, ensuring that features reflect the full session history rather than only the most recent window. This cumulative computation is appropriate for Hexad profiling (which requires sufficient behavioural evidence) while the `FlowClassifier` uses only the most recent window's features to classify the instantaneous state.

A notable implementation detail is the denominator protection pattern applied throughout: `max(deaths, 1)`, `max(obj_attempted, 1)`, etc. These guards prevent division by zero in the opening minutes of a session when many counters are still zero.

4.6 Narrative Pipeline Implementation

The narrative pipeline is invoked asynchronously following each player model update. The process follows four stages:

1. **Action selection:** The `NarrativeAgent.select_action()` method constructs the 7-dimensional observation vector from the `PlayerModel` and calls `model.predict(obs, deterministic=False)`.

2. **RAG retrieval:** The query string is constructed from the Flow state, current activity, and location to maximize retrieval relevance. The top-3 lore chunks are returned.

3. **Prompt assembly:** The `PromptBuilder` combines the system prompt, player context, retrieved lore, episodic memory context, and action-specific instructions.

4. **LLM generation:** The `OllamaClient` sends the prompt and returns the parsed `NarrativeContent`, with fallback handling for malformed or timed-out responses.

Stochastic prediction (`deterministic=False`) is used at inference time to maintain behavioural diversity; deterministic prediction would always select the maximum-probability action, potentially producing repetitive narrative patterns.

The importance score of 0.7 assigned to new memory events reflects a baseline estimate; a more sophisticated implementation could derive importance from the narrative action type (e.g., LORE_REWARD events receive higher importance than NO_CHANGE events).

4.7 RL Adaptation Implementation

The `NarrativeAdaptationEnv` is registered as a standard Gymnasium environment and trained using Stable-Baselines3 PPO. Table 4.3 lists the hyperparameters.

Table 4.3: PPO hyperparameters.

Hyperparameter	**Value**
Policy	MlpPolicy (2×64 hidden layers)
Learning rate	3×10^{-4}
n_steps	512
batch_size	64
n_epochs	10
gamma (γ)	0.95
gae_lambda	0.95
clip_range	0.2
n_envs (vectorized)	4
total_timesteps	200,000

Training uses four parallel vectorized environments (`make_vec_env`) to improve sample efficiency. The trained model is saved to `./data/ppo_narrative_agent.zip` and loaded on subsequent server starts, avoiding the training overhead at inference time. Episode length is 360 steps (one step per 5-second telemetry window, corresponding to 30 minutes of gameplay).

4.8 Unity Plugin Implementation

The Unity plugin is structured as a UPM-compatible package. The `PlayerDataLogger` accumulates telemetry via direct field increments in `Update()` and fires the HTTP POST at fixed 5-second intervals using a Coroutine.

The `NarrativeManager` uses a polling architecture: immediately following each telemetry POST's response, it fires a GET request to `/narrative/generate`. The 8-second Ollama generation timeout is handled by displaying no new content rather than blocking the game loop. The `ContentInjector` implements a state machine with three states: IDLE, DISPLAYING, and COOLING_DOWN. Content queued during COOLING_DOWN is held and injected after the 3-second minimum spacing, ensuring narrative delivery does not overwhelm the player during intense combat sequences.

For the demo game integration, additional scripts bridge the game engine's event system to the plugin's telemetry API: `TelemetryHooks.cs` connects combat, quest, and dialogue events to the corresponding `PlayerDataLogger` recording methods, while `ExplorationTracker.cs` implements a grid-based tile exploration counter that updates the `tiles_explored` field. `NarrativeUI.cs` provides the TMPro-based HUD panel with auto-dismiss and fade animations.

4.9 Testing and Validation

Unit tests were written using pytest covering: the `FeatureExtractor` (edge cases: empty batch list, zero-denominator fields, maximum value clamping), `HexadProfiler` (initialization with empty batches returns 0.5 defaults), `FlowClassifier` (boundary conditions at ratio 0.75 and 1.30, APATHY compound condition), and `compute_reward` (all state transitions, streak bonus triggering, repetition penalty).

Integration tests validated the end-to-end pipeline from a mock `TelemetryBatch` POST to a returned `NarrativeContent` JSON, confirming that Flow state correctly influences the generated tone and that the fallback mechanism activates when Ollama is unavailable.

Validation of the simulated MDP was conducted by verifying that the trained PPO agent converged to a mean episodic reward substantially above zero (indicating that the agent learned to prefer helpful actions) and that the agent's action distribution in ANXIETY states was dominated by PROVIDE_GUIDANCE and LOWER_STAKES — consistent with the design intention.

Lore retrieval validation was conducted by manually querying the retriever with ten representative in-session queries and verifying that retrieved chunks were semantically relevant. All ten manual queries returned at least one clearly relevant chunk in the top-3 results, confirming that the all-MiniLM-L6-v2 model provides adequate semantic discrimination for the lore corpus.

Chapter 5

RESULTS

5.1 Chapter Overview

This chapter presents the findings of the user study. All values marked as *[INSERT]* are structured placeholders that will be populated following data collection. The statistical tests and reporting structure are fully specified; only the numerical values remain to be inserted.

5.2 Participant Demographics

A total of $N = 50$ participants completed the study (25 Experimental, 25 Control). *[INSERT: exclusion details if applicable]* yielding a final sample of *[INSERT: N]*. Table 5.1 summarizes demographics.

Table 5.1: Participant demographics [PLACEHOLDER].

Variable	**Exp. (*n*=25)**	**Ctrl. (*n*=25)**	**Total (*N*=50)**
Age: Mean (SD)	*[INSERT]*	*[INSERT]*	*[INSERT]*
Gender: F / M / NB / PNS	*[INSERT]*	*[INSERT]*	*[INSERT]*
Gaming hours/week: Mean (SD)	*[INSERT]*	*[INSERT]*	*[INSERT]*
RPG experience (1–5): Mean (SD)	*[INSERT]*	*[INSERT]*	*[INSERT]*

No significant baseline differences between conditions were observed on age (t(*[INSERT]*) = *[INSERT]*, p = *[INSERT]*) or gaming hours (t(*[INSERT]*) = *[INSERT]*, p = *[INSERT]*), supporting the assumption of pre-intervention equivalence.

5.3 Quantitative Results

5.3.1 GEQ Flow Subscale (H1)

Table 5.2: GEQ Flow subscale descriptive statistics [PLACEHOLDER].

Condition	*n*	*M*	*SD*	**95% CI**
Experimental	25	*[INSERT]*	*[INSERT]*	*[INSERT]*
Control	25	*[INSERT]*	*[INSERT]*	*[INSERT]*

Welch's independent-samples t-test: t(*[INSERT]*) = *[INSERT]*, p = *[INSERT]*, Cohen's d = *[INSERT]*.

[INSERT: Narrative interpretation of H1 result.]

Shapiro-Wilk normality: Experimental W = *[INSERT]*, p = *[INSERT]*; Control W = *[INSERT]*, p = *[INSERT]*. Cronbach's α for GEQ Flow: *[INSERT]*.

5.3.2 GEQ Immersion Subscale (H2)

Table 5.3: GEQ Immersion subscale descriptive statistics [PLACEHOLDER].

Condition	n	M	SD	**95% CI**
Experimental	25	*[INSERT]*	*[INSERT]*	*[INSERT]*
Control	25	*[INSERT]*	*[INSERT]*	*[INSERT]*

Welch's t-test: t(*[INSERT]*) = *[INSERT]*, p = *[INSERT]*, Cohen's d = *[INSERT]*.

[INSERT: Narrative interpretation of H2 result. Cronbach's α for GEQ Immersion.]

5.3.3 Personalization Perception (H3)

Table 5.4: NPS-3 Personalization Scale descriptive statistics [PLACEHOLDER].

Condition	n	M	SD	**95% CI**
Experimental	25	*[INSERT]*	*[INSERT]*	*[INSERT]*
Control	25	*[INSERT]*	*[INSERT]*	*[INSERT]*

Welch's t-test: t(*[INSERT]*) = *[INSERT]*, p = *[INSERT]*, Cohen's d = *[INSERT]*.

[INSERT: Narrative interpretation of H3 result. NPS-3 Cronbach's α and item-level means.]

5.3.4 Behavioural Metrics

Table 5.5: Behavioural metrics comparison [PLACEHOLDER].

Metric	**Exp.** M (SD)	**Ctrl.** M (SD)	t	p	d
Voluntary NPC interactions	*[INSERT]*	*[INSERT]*	*[INSERT]*	*[INSERT]*	*[INSERT]*
Dialogue read ratio	*[INSERT]*	*[INSERT]*	*[INSERT]*	*[INSERT]*	*[INSERT]*
Exploration coverage	*[INSERT]*	*[INSERT]*	*[INSERT]*	*[INSERT]*	*[INSERT]*
miniPXI total score	*[INSERT]*	*[INSERT]*	*[INSERT]*	*[INSERT]*	*[INSERT]*

[INSERT: Exploratory analysis interpretation.]

5.4 Qualitative Results (Thematic Analysis)

Semi-structured interviews were conducted with *[INSERT: n]* participants. Following the six-phase Reflexive Thematic Analysis procedure (Braun and Clarke, 2022), *[INSERT: n]* initial codes were generated and refined into *[INSERT: n]* final themes.

Table 5.6: Qualitative themes and representative quotations [PLACEHOLDER].

Theme	**Description**	**Representative Quote**
[INSERT]	*[INSERT]*	*[INSERT]*
[INSERT]	*[INSERT]*	*[INSERT]*
[INSERT]	*[INSERT]*	*[INSERT]*
[INSERT]	*[INSERT]*	*[INSERT]*

5.5 Summary of Findings

[INSERT: A narrative summary of the overall pattern of results across quantitative and qualitative strands, addressing whether H1, H2, and H3 were supported, what the behavioural metrics indicate, and the dominant qualitative themes.]

Chapter 6

DISCUSSION

6.1 Chapter Overview

This chapter interprets the results presented in Chapter 5, connecting the findings to the research questions, objectives, and existing literature. It discusses whether the outcomes support or contradict prior studies, identifies the original contributions of the work, acknowledges limitations, and evaluates threats to validity.

6.2 Interpretation of Quantitative Findings

The central prediction of the research — that PPO-driven narrative adaptation would produce measurably higher Flow experience — rests on the theoretical chain from the adaptation engine through the player modelling module to the GEQ instrument. If the PPO agent successfully learns to apply narratively appropriate actions in response to Flow state signals, and if those narrative interventions succeed in shifting the player's psychological state toward FLOW, then the accumulated Flow experience across the session should register as a higher GEQ Flow subscale score.

[INSERT: Interpret the actual results when available. The following structures illustrate possible interpretations:]

If H1 is supported: The significant difference in GEQ Flow scores (d = *[INSERT]*) supports the core theoretical claim that real-time narrative personalization can improve Flow experience. The effect size is *[magnitude]* by Cohen's (1988) convention, suggesting that narrative adaptation is a meaningful lever for engagement even within a relatively short (30-minute) session. The convergent evidence from the behavioural metrics — specifically the dialogue read ratio in the Experimental condition — suggests that players were not merely reporting higher Flow retrospectively but were also demonstrably more engaged with the narrative content during play.

If H1 is not supported: The absence of a significant difference challenges the assumption that within-session narrative adaptation produces observable Flow differences within a 30-minute session. Several explanations merit consideration. First, the PPO agent was trained on a simulated environment whose transition dynamics only approximately match real player behaviour; policy transfer to the real game environment may be limited. Second, the 30-minute session may be insufficient for the player modelling module to accumulate enough telemetry. Third, the effect of narrative tone on Flow may be smaller than anticipated relative to mechanical game design factors.

[INSERT: Discuss H2 (Immersion) and H3 (Personalization) results in relation to each other and to H1.]

6.3 Themes from Interviews

[INSERT: Discuss qualitative themes in relation to the quantitative results and existing literature.]

The interview data are expected to illuminate the mechanisms by which narrative adaptation influenced (or failed to influence) experience. A theme of **narrative responsiveness** — players describing the story as attuned to their behaviour — would provide direct qualitative support for the personalization hypothesis (H3). Conversely, a theme of **artificiality awareness** — players noticing repetitive patterns, inconsistent tone, or implausible content — would identify a key failure mode of the LLM generation pipeline.

The Flow-related interview questions are expected to reveal whether narrative injections were perceived as disruptive or supportive of engagement. A recurring finding in studies of game storytelling is that narrative interruptions during high-engagement moments are perceived negatively even when the content is high quality. The `ContentInjector`'s 3-second spacing mechanism was designed to mitigate this, but interview data will reveal whether it was sufficient.

6.4 Novelty and Contributions

This work makes the following original contributions:

Contribution 1: Telemetry-derived Hexad profiling. Prior work on Hexad-based game adaptation has relied exclusively on pre-play questionnaire scores (Tondello et al., 2016). The present system computes six-dimensional continuous Hexad profiles from behavioural telemetry, enabling within-session updating without any player self-report. The mapping from telemetry signals to Hexad dimensions (Table 3.3) provides a reusable operational template for game developers.

Contribution 2: RAG-grounded dynamic narrative. While PANGeA (Todd et al., 2024) demonstrated LLM-based procedural narrative grounded in prompt-delivered context, the present work extends this to a dynamic retrieval paradigm in which the specific lore passages delivered to the LLM are selected at runtime based on the player's current context. This ensures semantic relevance to the player's location and activity.

Contribution 3: Episodic session memory in the LLM prompt. The implementation of importance-weighted episodic memory inspired by EM-LLM (Fountas et al., 2024) enables narrative continuity across a play session. To the author's knowledge, this is the first application of episodic memory compression to real-time game narrative generation.

Contribution 4: PPO for narrative action selection. Reinforcement learning has been applied extensively to mechanical game adaptation (DDA) but not, to the author's knowledge, to narrative action selection. Framing narrative tone selection as an RL problem represents a meaningful conceptual advance over rule-based narrative adaptation systems.

Contribution 5: Flow-based MDP reward. The reward function directly operationalizes Csikszentmihalyi's (1990) challenge–skill balance theory, providing a principled theoretical grounding for the adaptation objective.

Contribution 6: Complete closed-loop integration. The five-module architecture is novel in its end-to-end integration: from raw telemetry capture in Unity, through behavioural feature extraction and player typing, through RL action selection, through RAG-grounded LLM generation, to in-game content injection — all within a single deployable plugin.

6.5 Limitations

Several limitations merit acknowledgment.

Simulation fidelity. The PPO agent is trained on a simulated environment with hand-specified transition probabilities ($p = 0.75$ for successful helpful actions). Real player behaviour is substantially more complex; the simulation's Markovian assumption is almost certainly violated in practice, as player engagement trajectories have temporal dependencies extending well beyond the current state.

Rule-based Hexad profiling. The mapping from telemetry signals to Hexad dimensions is based on theoretical alignment rather than empirical validation against questionnaire-derived ground truth. Players may exhibit behavioural patterns that are systematically misclassified — for instance, a player who moves slowly due to hardware input lag rather than deliberate exploration will register high `explore_rate` and be profiled as an Explorer.

LLM generation quality. Phi-3.5 Mini (3.8B parameters) is a small language model constrained by its capacity relative to larger frontier models. The quality of generated narrative — its fluency, creativity, and character voice consistency — is limited by the model size.

Single-session study. The study captures a single 30-minute session per participant. Long-term effects of narrative personalization cannot be assessed within this design.

Testbed ecological validity. The testbed game is a simplified prototype. Findings may not generalize to commercially developed games with more complex narrative architectures.

Sample size. With $n = 25$ per condition, the study is powered to detect medium effects ($d \approx 0.5$) but not small effects. The convenience sample also limits demographic diversity.

6.6 Threats to Validity

Internal validity. The primary threat is demand characteristics: participants who infer they are in the experimental condition may respond more favourably. The blind design mitigates but does not eliminate this threat.

Construct validity. The GEQ Flow subscale measures retrospective self-report of flow, not flow during gameplay. The absence of a physiological measure limits confidence that the score reflects the within-session psychological state.

External validity. The testbed game and student convenience sample limit generalizability.

Statistical conclusion validity. With $N = 50$, the study is not robustly powered for small effect sizes, and Bonferroni correction reduces power further.

Chapter 7

CONCLUSION

7.1 Summary

This dissertation has presented a complete design, implementation, and evaluation methodology for an AI-driven closed-loop narrative personalization plugin for Unity games. The system integrates five modules — telemetry collection, player modelling, narrative generation, RL adaptation, and content injection — into a coherent pipeline that operates within 5-second adaptation cycles.

The six principal contributions are: (1) a method for deriving continuous Hexad player-type profiles from behavioural telemetry without requiring a player questionnaire; (2) a RAG-grounded narrative generation pipeline using sentence-transformers cosine retrieval over a structured lore corpus; (3) an importance-weighted episodic session memory that provides LLM narrative continuity across a play session; (4) a PPO reinforcement learning agent trained to select among eight narrative tone actions based on player psychological state; (5) a Flow-based MDP reward function operationalizing Csikszentmihalyi's challenge–skill balance theory; and (6) a complete closed-loop integration of all five components within a deployable Unity plugin.

The evaluation design — a between-subjects study ($N = 50$) using the GEQ, miniPXI, and custom NPS-3 scale, supplemented by reflexive thematic analysis of post-play interviews — provides a rigorous mixed-methods framework for assessing the system's impact on Flow experience, immersion, and perceived personalization.

7.2 Limitations

The key limitations of this work, discussed in detail in Section 6.4, include: the reliance on a simulated MDP for PPO training rather than real player data; the rule-based (rather than empirically validated) Hexad profiling; the constrained generation quality of the 3.8B-parameter LLM; the single-session study design; the simplified testbed game; and the modest sample size. These limitations are inherent to the scope of an undergraduate research project and define clear directions for future work.

7.3 Future Work

Several directions for future work present themselves.

Empirical validation of telemetry-derived Hexad profiling. The most immediate extension is a validation study comparing telemetry-derived Hexad scores against matched questionnaire scores from the same participants, quantifying the construct validity of the behavioural proxy.

Online RL adaptation. An online or hybrid approach — in which the agent continues to update its policy based on real player interaction data — would produce a policy increasingly tailored to the real game environment. Careful management of exploration–exploitation trade-offs would be necessary.

Larger LLMs and fine-tuning. Replacing Phi-3.5 Mini with a larger model (e.g., Llama 3.1 8B or a fine-tuned narrative generation model) could substantially improve generation quality and character voice consistency.

Multi-session study. A longitudinal study spanning three to five sessions per participant would reveal whether the personalization system produces cumulative benefits.

Evaluation in a commercial game context. Collaborating with an independent game developer to deploy the plugin in an in-development commercial title would provide a more ecologically valid test.

Voice and audio integration. Integrating text-to-speech generation with character-specific voice parameters would create a richer multimodal experience.

Explainability and player agency. A transparency mode in which players can see why a particular narrative action was chosen could be studied as a mechanism for building player trust.

7.4 Reflective Report

This section provides a personal reflection on the research process, challenges faced, and lessons learned throughout the study.

7.4.1 Project Management and Planning

The project spanned approximately seven months, from initial literature review in September 2025 to final submission in April 2026. Early planning focused on defining the system architecture and identifying the key technical risks. The decision to use a locally hosted LLM (Ollama with Phi-3.5 Mini) rather than a cloud API was made early and proved to be one of the most consequential design choices: it eliminated network dependency and data governance concerns but introduced generation quality constraints that influenced the prompt engineering effort throughout the project.

Time management was a persistent challenge. The implementation phase took longer than anticipated, primarily due to the complexity of integrating five distinct modules with different technology stacks (Python backend, C# Unity plugin, Ollama inference server). Weekly logbook entries helped maintain accountability and provided a structured record of decisions and progress.

7.4.2 Technical Challenges and Learning

Several technical challenges required significant problem-solving effort. First, the original plan called for ChromaDB as the vector database for RAG, but Python 3.14 compatibility issues with ChromaDB's Pydantic v1 dependency necessitated a pivot to in-memory numpy-based cosine similarity search. This experience reinforced the importance of verifying dependency compatibility early in the project lifecycle.

Second, designing the reward function for the PPO agent required iterating through several formulations. Early versions without the repetition penalty produced degenerate policies that repeatedly selected the same action regardless of state. Adding the penalty and the shaped rewards (streak bonus, transition bonus) was informed by reinforcement learning literature on reward shaping and required careful tuning to produce stable training.

Third, ensuring that the LLM produced valid JSON output consistently was more challenging than anticipated. The combination of JSON format mode, explicit schema instructions in the prompt, and a parsing-with-fallback pipeline was the result of extensive trial and error.

7.4.3 Research Skills Development

This project significantly developed my research skills across several dimensions. Designing the mixed-methods evaluation — including selecting validated instruments, specifying hypotheses, planning statistical analyses, and preparing an interview guide — provided practical experience with research methodology that extended well beyond the technical implementation work. Learning to apply Reflexive Thematic Analysis (Braun and Clarke, 2022) and to plan for Bonferroni-corrected multiple comparisons were particularly valuable methodological skills.

The literature review process taught me to read critically rather than summatively: to identify not just what previous work achieved but what it omitted, and to position my own contributions relative to those gaps. The discipline of maintaining a structured gap analysis table (Table 2.1) proved invaluable for articulating the novelty of the present work.

7.4.4 Ethical Awareness

Working with human participants heightened my awareness of ethical responsibilities in research. Designing the informed consent process, ensuring data anonymization, and planning the debrief procedure required careful consideration of participant welfare. The question of whether AI-driven narrative personalization constitutes a form of behavioural manipulation — and how to communicate this transparently to participants — is an ethical dimension that I had not fully considered at the outset of the project.

7.4.5 Key Takeaways

If I were to undertake a similar project again, I would make three changes. First, I would conduct a small-scale pilot study earlier in the project timeline to identify usability issues and calibrate the telemetry thresholds before committing to the full evaluation design. Second, I would invest more time in automated integration testing between the Python backend and Unity client, as manual testing of the WebSocket connection consumed considerable debugging time. Third, I would begin the literature review with a more systematic search strategy (e.g., using PRISMA guidelines) rather than the snowball approach I initially adopted, as the systematic approach would have surfaced relevant work more efficiently.

Overall, this project has been the most technically ambitious and intellectually demanding work of my undergraduate degree. It required synthesizing knowledge from multiple domains — reinforcement learning, natural language processing, game design, psychometrics, and research methodology — into a single coherent system. The experience has confirmed my interest in pursuing research at the intersection of AI and interactive media and has equipped me with the technical and methodological skills to do so.

7.5 Closing Remarks

The aspiration behind this research is modest in scope but substantive in ambition: to demonstrate that the tools now available to AI researchers — large language models, reinforcement learning, semantic retrieval — can be composed into a practical system that makes games more attentive to the people who play them. The challenge is not merely technical but also philosophical: what does it mean for a game's story to understand a player? What behavioural signals are worth attending to, and what should be ignored? When does adaptation feel like personalization and when does it feel like surveillance?

These questions are ultimately design and ethics questions as much as they are engineering questions, and the present work represents only an early step toward answering them. The closed-loop system described here is a proof of concept: a demonstration that the integration is feasible, that the components can communicate, and that the adaptation cycle can run within the latency constraints of real-time gameplay. Whether it produces experiences that players find genuinely enriched by the adaptation — whether the personalization is felt rather than merely inferred — is an empirical question that the evaluation study will begin to answer.

The long arc of games as a medium bends toward greater responsiveness to individual players. Dynamic narrative personalization, grounded in psychological models and driven by learned adaptive policies, is one promising path along that arc.

Bibliography

Bartle, R. (1996). Hearts, clubs, diamonds, spades: Players who suit MUDs. *Journal of MUD Research*, 1(1). Available at: https://mud.co.uk/richard/hcds.htm.

Braun, V. and Clarke, V. (2022). *Thematic Analysis: A Practical Guide*. SAGE Publications.

Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., and Zaremba, W. (2016). OpenAI Gym. *arXiv preprint arXiv:1606.01540*.

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems (NeurIPS)*, 33:1877–1901.

Chen, J. (2007). Flow in games (and everything else). *Communications of the ACM*, 50(4):31–34.

Cohen, J. (1988). *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, 2nd edition.

Compton, K., Kybartas, B., and Mateas, M. (2015). Tracery: An author-focused generative text tool. In *Interactive Storytelling: 8th International Conference on Interactive Digital Storytelling*, pages 154–161. Springer.

Creswell, J. W. and Plano Clark, V. L. (2018). Designing and conducting mixed methods research.

Csikszentmihalyi, M. (1990). *Flow: The Psychology of Optimal Experience*. Harper & Row.

Faul, F., Erdfelder, E., Lang, A.-G., and Buchner, A. (2007). G*Power 3: A flexible statistical power analysis program for the social, behavioral, and biomedical sciences. *Behavior Research Methods*, 39(2):175–191.

Fountas, Z., Sylaidi, A., Bhatt, U., and Bhatt, P. (2024). Human-like episodic memory for infinite context LLMs. *arXiv preprint arXiv:2407.09450*. Accepted at ICLR 2025.

Hamari, J., Koivisto, J., and Sarsa, H. (2014). Does gamification work? A literature review of empirical studies on gamification. In *Proceedings of the 47th Hawaii International Conference on System Sciences*, pages 3025–3034. IEEE.

Hunicke, R. and Chapman, V. (2004). AI for dynamic difficulty adjustment in games. In *Proceedings of the AAAI Workshop on Challenges in Game AI*, pages 91–96. AAAI Press.

IJsselsteijn, W. A., de Kort, Y. A. W., and Poels, K. (2013). The Game Experience Questionnaire. Technical report, Technische Universiteit Eindhoven, Eindhoven.

Kreminski, M., Samuel, B., Melcer, E., and Wardrip-Fruin, N. (2019). Evaluating AI-based games through the lens of interactivity. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, pages 59–65. AAAI Press.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474.

Lopes, R. and Bidarra, R. (2011). Adaptivity challenges in games and simulations: A survey. *IEEE Transactions on Computational Intelligence and AI in Games*, 3(2):85–99.

Mateas, M. and Stern, A. (2003). Façade: An experiment in building a fully-realized interactive drama. In *Proceedings of the Game Developers Conference (GDC)*. CMP Media.

Microsoft (2024). Phi-3.5 Mini Instruct model card. Microsoft Research, Hugging Face. Available at: https://huggingface.co/microsoft/Phi-3.5-mini-instruct.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533.

Nakamura, J. and Csikszentmihalyi, M. (2002). The concept of flow. In Snyder, C. R. and Lopez, S. J., editors, *Handbook of Positive Psychology*, pages 89–105. Oxford University Press.

Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., and Dormann, N. (2021). Stable-Baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8.

Riedl, M. O. and Bulitko, V. (2013). Interactive narrative: An intelligent systems approach. In *AI Magazine*, volume 34, pages 67–77.

Ryan, R. M. and Deci, E. L. (2000). Self-determination theory and the facilitation of intrinsic motivation, social development, and well-being. *American Psychologist*, 55(1):68–78.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.

Short, E. (2019). Procedural generation and the problem of authorial intent. In *Proceedings of Foundations of Digital Games (FDG 2019)*. ACM.

Todd, G., Earle, S., Nasir, M. U., Green, M. C., and Togelius, J. (2024). PANGeA: Procedural artificial narrative using generative AI. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE 2024)*. AAAI Press.

Tondello, G. F., Wehbe, R. R., Diamond, L., Busch, M., Marczewski, A., and Nacke, L. E. (2016). The gamification user types Hexad scale. In *Proceedings of the Annual Symposium on Computer-Human Interaction in Play (CHI PLAY 2016)*, pages 229–243. ACM.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 30.

Vornhagen, J. B., Thue, D., and Göbel, S. (2022). miniPXI: Development and validation of an eleven-item measure of player experience. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems (CHI 2022)*. ACM.

Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., and Zhou, M. (2020). MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 5776–5788.

Yannakakis, G. N. and Hallam, J. (2009). Real-time game adaptation for optimizing player satisfaction. *IEEE Transactions on Computational Intelligence and AI in Games*, 1(2):121–133.

Yee, N. (2006). Motivations for play in online games. *CyberPsychology & Behavior*, 9(6):772–775.

Zhu, J., Ontanon, S., and Earle, S. (2023). Player modeling for interactive narrative: A survey. In *IEEE Transactions on Games*, volume 15, pages 514–528.

Chapter A

CODE REPOSITORY

The complete source code for this project is available at the following repository:

`https://github.com/Rivi9/dynamic-quest-gen`

The repository contains:

- `backend/` — FastAPI Python backend (player modelling, narrative generation, RL adaptation)
- `unity-plugin/` — UPM package (telemetry collection, narrative management, content injection)
- `demo-game/` — Unity 6 testbed project
- `lore/` — Game world lore documents (RAG corpus)
- `evaluation/` — Questionnaires, interview guide, and statistical analysis scripts
- `data/` — Trained PPO model weights and SQLite database

Note: The full source code is not reproduced in this thesis per submission guidelines. Please refer to the repository for implementation details.

Chapter B

GEQ CORE MODULE

The Game Experience Questionnaire Core Module as administered in this study. Reproduced from IJsselsteijn et al. (2013) for research purposes.

Scale: 0 = Not at all, 1 = Slightly, 2 = Moderately, 3 = Fairly, 4 = Very much

Please answer each question based on your experience during the game session you just played.

#	Item	Scale
1	I felt content	0–4
2	I felt skillful	0–4
3	I was absorbed in the game	0–4
4	I felt happy	0–4
5	This game gave me a bad mood	0–4 (R)
6	I thought it was fun	0–4
7	I was fully occupied with the game	0–4
8	I felt annoyed	0–4 (R)
9	I found the game satisfying	0–4
10	I felt frustrated	0–4 (R)
11	It felt like a whole new world	0–4
12	I lost track of time	0–4
13	I felt bored	0–4 (R)
14	I felt good	0–4
15	I was good at this game	0–4
16	I felt tired	0–4 (R)
17	I felt imaginative	0–4
18	I felt that I could explore things	0–4
19	I felt like I had to do things quickly	0–4
20	I had fun with others	0–4
21	I felt challenged	0–4
22	I felt that time went by quickly	0–4
23	I felt pressured	0–4 (R)
24	I felt competent	0–4
25	I felt engrossed	0–4
26	I felt bothered by aspects of the game	0–4 (R)

#	Item	Scale
27	I was deeply concentrated in the game	0–4
28	I forgot everything around me	0–4
29	I felt pleased	0–4
30	I felt completely focused on the game	0–4
31	I enjoyed the game	0–4
32	I felt imaginative	0–4
33	I felt adventurous	0–4

(R) = reverse-scored

Subscale scoring:

Subscale	Items	Notes
Flow (Primary DV, H1)	5(R), 13(R), 25, 28, 31	Mean of 5 items
Immersion (Secondary DV, H2)	3, 12, 18, 19, 27, 30	Mean of 6 items
Positive Affect	1, 4, 8(R), 20, 29	Mean of 5 items
Negative Affect	10, 16, 23, 26	Mean of 4 items
Tension	7, 22, 25	Mean of 3 items
Competence	2, 15, 17, 21, 24	Mean of 5 items

Chapter C

MINIPXI QUESTIONNAIRE

Citation: Vornhagen et al. (2022)

Scale: 1 = Strongly Disagree . . . 7 = Strongly Agree

#	**Item**
1	I felt in control of my character/avatar
2	I felt a sense of mastery when completing challenges
3	I found the game world interesting to explore
4	I was curious about what would happen next in the game
5	I felt like the game narrative pulled me in
6	I had a sense of presence in the game world
7	I felt free to make meaningful choices
8	The game felt meaningful to me
9	I had positive emotions while playing
10	I felt excited during the game
11	Overall, I enjoyed playing this game

Subscales: Mastery (1, 2); Curiosity (3, 4); Immersion (5, 6); Autonomy (7, 8); Positive Affect (9, 10, 11). Total score: mean of all 11 items.

Chapter D

INTERVIEW GUIDE

Study: AI-Driven Dynamic Narrative Personalization in Games
Duration: $\sim$20 minutes
Analysis method: Reflexive Thematic Analysis (Braun and Clarke, 2022)

Before the interview: Obtain signed consent; remind participant there are no right or wrong answers; start audio recording.

Section 1 — General Experience (5 min)

1. Can you walk me through what you remember most about the game experience?
2. How would you describe the story and the way it unfolded for you?
3. Were there any moments that stood out to you, positively or negatively?

Section 2 — Narrative and Personalization (8 min)

4. Did the narrative feel connected to the way you were playing? Can you give an example?
5. Were there moments where the story surprised you — or felt unexpectedly relevant?
6. Did any NPCs or dialogue feel particularly fitting or particularly out of place?
7. *(Experimental only)* Did you notice the story adapting to you? How did that feel?
8. *(Control only)* How well did the story match your play style?

Section 3 — Flow and Engagement (5 min)

9. Were there moments where you felt in "the zone" — neither too stressed nor too bored?
10. Were there moments where the game felt too easy, too hard, or just right?
11. How did the story contribute to those moments of engagement?

Section 4 — Closing (2 min)

12. Is there anything about the game or narrative you would like to add?
13. Any suggestions for improving the experience?

Probes: "Can you say more about that?" / "What do you mean when you say [X]?" / "When did that happen in the game?" / "How did that make you feel?"

Chapter E

API SPECIFICATION

The FastAPI backend exposes the following REST and WebSocket endpoints:

Method	Path	Description
POST	/telemetry/batch	Submit telemetry window; returns PlayerModel
GET	/narrative/generate	Retrieve generated narrative for session
GET	/player/model/{player_id}	Get current PlayerModel
GET	/player/hexad/{player_id}	Get current HexadProfile
DELETE	/session/{session_id}	Clear session data
WebSocket	/ws/telemetry/{session_id}	Low-latency telemetry stream

NarrativeContent response example:

```
{
  "action_taken": "PROVIDE_GUIDANCE",
  "content_type": "dialogue",
  "content": "The path below is treacherous. If you
    still carry the torch, keep it lit.",
  "speaker": "Commander Varis",
  "emotional_tone": "tense",
  "lore_refs": [],
  "fallback": false
}
```

All endpoints return standard HTTP status codes. 422 for malformed requests (Pydantic validation). 503 if Ollama is unreachable (returns fallback NarrativeContent with `"fallback": true`).

Chapter F

MDP FORMAL SPECIFICATION

Formal MDP tuple: (S, A, T, R, γ)

State space S: Box(7,) $\in [0,1]^7$

Dim	Symbol	Range	Description
s_0	flow_score	{0.1, 0.2, 0.3, 1.0}	Encoded Flow state
s_1	csr_norm	[0, 1]	challenge_skill_ratio / 3.0
s_2	h_{explorer}	[0, 1]	Hexad Explorer score
s_3	$h_{\text{socializer}}$	[0, 1]	Hexad Socializer score
s_4	h_{achiever}	[0, 1]	Hexad Achiever score
s_5	$h_{\text{disruptor}}$	[0, 1]	Hexad Disruptor score
s_6	t_{progress}	[0, 1]	session_elapsed / 1800

Reward function $R(s, a, s')$:

$$R = R_{\text{state}}(s') + R_{\text{step}} + R_{\text{streak}}(s', n) + R_{\text{transition}}(s, s') + R_{\text{repetition}}(a, a_{[-3:]})$$

Where:

- R_{state}: FLOW $= +1.0$, BOREDOM $= -0.3$, ANXIETY $= -0.5$, APATHY $= -0.8$
- $R_{\text{step}} = -0.05$
- $R_{\text{streak}} = +0.20$ if $s' =$ FLOW and $n > 2$
- $R_{\text{transition}} = +0.30$ if $s \in \{\text{ANXIETY}, \text{BOREDOM}\}$ and $s' =$ FLOW
- $R_{\text{repetition}} = -0.15$ if all last 3 actions identical

Discount factor γ: 0.95

Episode length: 360 steps (30 minutes at 5-second intervals)

Training: PPO (Stable-Baselines3 v2.x), 200,000 timesteps, 4 parallel environments.